Registros de decisión y taller de diagramas

Unidad 3 · Diseño de software · SIS3407

La sesión pasada vimos cómo se le enseña un sistema a otra persona. Hoy va lo otro, que casi nunca se enseña y siempre hace falta: **por qué el sistema quedó así.** Un diagrama responde qué hay; un registro de decisión responde por qué. Y al final de la clase vas a entregar las dos cosas.

LA PREGUNTA QUE NADIE PUEDE CONTESTAR

En marzo entra alguien nuevo al proyecto, abre el repositorio y se encuentra un monolito. Piensa lo que piensa cualquiera que acaba de llegar: *esto está mal hecho, debería ser microservicios.* Lo parte en tres servicios, y en dos semanas se topa con exactamente los mismos problemas que tú ya habías pensado en octubre: nadie en el equipo sabe operar servicios en red, los despliegues se multiplicaron y los errores ahora aparecen en tres lugares distintos.

El problema no fue su decisión. **El problema fue que tu razonamiento no existía en ningún lado.** La discusión duró dos horas, fue buena, consideró lo correcto, y se evaporó. Lo único que quedó fue el resultado, sin el porqué, y un resultado sin su porqué parece arbitrario.

> Esto tiene un nombre y es muy común: se llama **conocimiento tribal**. Es todo lo que el equipo sabe pero que no está escrito en ningún lado, y que por lo tanto se va con la persona que se va.

QUÉ ES UN REGISTRO DE DECISIÓN DE ARQUITECTURA

Un registro de decisión de arquitectura, o **ADR** por sus siglas en inglés (Architecture Decision Record), es **un archivo de texto, corto, uno por decisión, que vive junto al código y explica por qué se decidió lo que se decidió.**

Fíjate en lo que no es. No es un documento de arquitectura, no describe el sistema completo y no se actualiza cada vez que algo cambia. Es una bitácora: se escribe una vez, cuando se toma la decisión, y se queda ahí.

> **Imagínatelo así.** El expediente clínico de un paciente de la clínica. El expediente no describe cómo está hecho el perro; eso ya se sabe. Lo que anota es qué se le hizo, cuándo y por qué: que se le dio este antibiótico porque el anterior le cayó mal, que no se le operó porque la edad no lo permitía.
>
> El siguiente veterinario que lo atienda no necesita adivinar. Y sobre todo, **no repite el tratamiento que ya falló.** Eso es un ADR para un sistema.

La idea la propuso **Michael Nygard en 2011**, en un artículo que se llama justamente *Documenting Architecture Decisions*, y la plantilla que propuso ahí es la que se sigue usando hoy. Son cinco secciones y caben en una página.

LAS CINCO SECCIONES

Título | Contexto | Decisión | Estado | Consecuencias

Título

Qué va ahí	Un número y una frase corta que diga cuál es la decisión, no cuál es el tema. ADR-003 y luego la frase.
El error típico	Poner el tema en lugar de la decisión. Arquitectura del sistema no dice nada; alguien que busca en la lista no sabe si eso le sirve o no.
En la clínica	ADR-003: Monolito modular en lugar de microservicios.

Un ADR completo
Así se ve el archivo entero, y sí, es todo lo que lleva

Copiar

```
# ADR-003: Monolito modular en lugar de microservicios

## Estado
Aceptada, 29 de octubre de 2026

## Contexto
El sistema de la clinica tiene tres modulos: agenda, cobros y
expedientes. El equipo es de cuatro personas y ninguna ha operado
servicios en red. La clinica tiene una sola sucursal y atiende
alrededor de 60 citas al dia.

Se evaluo separar cobros en un servicio aparte, porque es el modulo
que mas cambia y el que toca dinero.

## Decision
Construimos un monolito modular: un solo desplegable, con los tres
modulos separados por carpetas e interfaces, y con la regla de que
ningun modulo lee las tablas de otro.

## Consecuencias
Un solo despliegue y un solo registro de errores, que es lo que el
equipo si puede operar hoy.

No podemos escalar cobros por separado. Con 60 citas al dia no hace
falta, pero si la clinica abre sucursales habra que revisarlo.

Si algun dia hay que separarlo, los limites entre modulos ya estan
puestos y el corte es por donde ya cortamos.

El riesgo real es que alguien lea las tablas de otro modulo por
atajo y se pierda el limite. Lo revisamos en cada pull request.
```

Léelo completo. No hay una sola palabra de relleno, y aun así cualquiera que llegue en marzo entiende por qué el sistema quedó así y bajo qué condiciones dejaría de tener sentido.

La regla que casi nadie respeta

Cuando una decisión se reemplaza, **el registro viejo no se borra.** Se le cambia el estado a "Reemplazada por ADR-007" y se queda ahí para siempre.

Parece contraintuitivo, porque lo normal sería limpiar lo que ya no aplica. Pero el valor del archivo no es saber qué decidimos hoy; eso se ve en el código. **El valor es saber qué ya intentamos y por qué dejamos de hacerlo.** Si borras el ADR viejo, borras justo esa parte, y en dos años alguien va a proponer otra vez lo que ya se probó y no funcionó.

DÓNDE VIVE EL ARCHIVO

La convención es una carpeta dentro del repositorio, con los archivos numerados:

Estructura
Dentro de tu propio repositorio

```
doc/
  arch/
    adr-001-usamos-postgresql.md
    adr-002-autenticacion-con-tokens.md
    adr-003-monolito-modular.md
```

La pregunta razonable es por qué ahí y no en un documento en la nube de alguien. Por tres razones concretas: **viaja con el código**, así que quien clona el repositorio ya tiene las decisiones; **se versiona con el código**, así que puedes ver cuándo se escribió y quién; y **se revisa en el mismo pull request** que el cambio que describe, en lugar de quedar para después, que en la práctica significa nunca.

EJERCICIO · ¿EN QUÉ SECCIÓN VA?

Cuatro frases sacadas de una discusión de equipo. Decide en qué sección del ADR va cada una.

FRASE 1

"La clínica tiene una sola sucursal y atiende alrededor de 60 citas al día."

Contexto | Decisión | Consecuencias | En ninguna

Correcto. Es contexto: son los hechos de la situación, con números, que hacen que la decisión tenga sentido. Y fíjate que es justo el dato que alguien en marzo necesita para saber si las condiciones cambiaron.

FRASE 2

"Ningún módulo lee las tablas de otro; se piden las cosas por la interfaz."

Contexto | Decisión | Consecuencias | En ninguna

Correcto. Es la decisión: es una regla que el equipo escogió seguir y que pudo haber sido distinta. En voz activa, en presente y sin rodeos, que es como se escribe esta sección.

FRASE 3

"No vamos a poder escalar cobros por separado si algún día hace falta."

Contexto | Decisión | Consecuencias | En ninguna

Correcto, y es una consecuencia negativa, que son las que más cuestan de escribir y más valen. Un ADR sin consecuencias negativas es un ADR que no analizó nada.

FRASE 4

"Los microservicios son la arquitectura más moderna, así que conviene usarlos."

Contexto | Decisión | Consecuencias | En ninguna

Correcto, y esta era la trampa. No es un hecho, no es una decisión y no es una consecuencia: es una opinión sin dato detrás. Más moderno no es un criterio de arquitectura. Si esa frase es el argumento más fuerte que tienes, todavía no hay decisión que documentar.

SEGUNDA PARTE

Taller de diagramas con Mermaid

Ya sabes qué dibujar y por qué dejar escrito el porqué. Falta la herramienta, y es más simple de lo que parece.

QUÉ ES MERMAID Y POR QUÉ TEXTO EN LUGAR DE DIBUJO

Mermaid es una herramienta donde **escribes el diagrama como texto y ella lo dibuja por ti.** No arrastras cajas, no acomodas flechas y no peleas con la alineación: declaras qué cajas hay y quién apunta a quién, y la herramienta se encarga del acomodo.

La razón de fondo no es la comodidad, aunque se agradece. Es que **un diagrama que es texto se puede versionar igual que el código.** Una imagen no: si alguien cambia el diagrama, lo único que ves en el repositorio es que el archivo pesa distinto. Si el diagrama es texto, ves exactamente qué caja se agregó y qué flecha se quitó, y eso se puede revisar y discutir en un pull request como cualquier otro cambio.

> Esto conecta con algo de la sesión pasada. C4 es **independiente de la notación y de la herramienta**, y eso es justo lo que nos deja usar Mermaid: no hay símbolos oficiales que respetar, así que cajas y flechas bien etiquetadas son C4 perfectamente válido.

LO MÍNIMO QUE NECESITAS SABER

`flowchart TD`	Abre el diagrama, de arriba hacia abajo. Si lo prefieres horizontal, `flowchart LR`, de izquierda a derecha.
`api[API del sistema]`	Una caja. Lo de la izquierda es el identificador con el que te refieres a ella; lo de los corchetes es el texto que se ve.
`db[(Base de datos)]`	Un cilindro, la forma típica de un almacén de datos.
`rec([Recepcionista])`	Una caja de puntas redondeadas. Útil para las personas que usan el sistema.
`web --> api`	Una flecha de la primera caja a la segunda.
`web -->|HTTPS| api`	La misma flecha, con una etiqueta encima. Esto es lo que vuelve útil un diagrama: la flecha sola no dice nada.
`App web React`	Un salto de línea dentro de una caja. Sirve para poner el nombre arriba y la tecnología abajo.

`subgraph s[Sistema]`
`  ...`
`end`
	Agrupa varias cajas dentro de un marco. Es lo que usas para dibujar la frontera de tu sistema.

`style api fill:#ff5d00,color:#fff`	Pinta una caja. Úsalo para resaltar la pieza de la que estás hablando, no para decorar.

LOS TRES DIAGRAMAS, LISTOS PARA PEGAR

Abre **mermaid.live**, borra lo que traiga de ejemplo y pega cada uno. Vas a ver el dibujo aparecer del lado derecho mientras escribes. Si quieres repasar las herramientas del curso, están en la hoja de herramientas que ya publicamos en la plataforma.

Nivel 1 · Diagrama de contexto
El sistema completo, quién lo usa y con quién habla

Copiar

```
flowchart TD
    rec([Recepcionista])
    vet([Veterinario])
    sis[Sistema de la clinica]
    lab[Laboratorio externo]
    pago[Pasarela de pagos]

    rec -->|agenda y cobra| sis
    vet -->|consulta expedientes| sis
    sis -->|envia estudios| lab
    sis -->|cobra consultas| pago

    style sis fill:#ff5d00,color:#fff
```

Fíjate en que no hay una sola palabra de tecnología. Este es el diagrama que le puedes enseñar al dueño de la clínica.

Nivel 2 · Diagrama de contenedores
Las piezas que se despliegan por separado

Copiar

```
flowchart TD
    rec([Recepcionista])

    subgraph sistema[Sistema de la clinica]
        web[App web<br/>React]
        api[API<br/>Node.js]
        db[(Base de datos<br/>PostgreSQL)]
    end

    lab[Laboratorio externo]

    rec -->|usa| web
    web -->|HTTPS/JSON| api
    api -->|lee y escribe| db
    api -->|SFTP| lab
```

```
    style api fill:#ff5d00,color:#fff
```

Aquí sí aparece la tecnología, y el `subgraph` es lo que dibuja la frontera: lo de adentro es tu sistema, lo de afuera no.

Nivel 3 · Diagrama de componentes
Lo que hay dentro de un contenedor

Copiar

```
flowchart TD
    web[App web]

    subgraph api[API del sistema]
        agenda[Agenda<br/>citas y horarios]
        cobros[Cobros<br/>cuentas y pagos]
        exp[Expedientes<br/>historial clinico]
        notif[Notificador<br/>interfaz de envio]
    end

    db[(Base de datos)]
    correo[Servicio de correo]

    web --> agenda
    web --> cobros
    web --> exp
    cobros -->|pregunta que se atendio| agenda
    cobros -->|enviarRecibo| notif
    notif --> correo
    agenda --> db
    cobros --> db
    exp --> db

    style cobros fill:#ff5d00,color:#fff
```

Mira dos cosas. Cobros le **pregunta** a Agenda en lugar de leer sus tablas, que es la regla del ADR de arriba hecha dibujo. Y el Notificador está puesto como interfaz, no como "Gmail", que es el Adapter de la semana pasada hecho dibujo.

LO QUE ROMPE EL DIAGRAMA Y LO QUE NO

- × **Paréntesis sueltos dentro de los corchetes.** `api[API (Node.js)]` truena. Si necesitas paréntesis, mete el texto entre comillas: `api["API (Node.js)"]`.
- × **Identificadores con espacios.** `mi nodo[Texto]` truena. El identificador va pegado: `minodo[Texto]`.
- × **La flecha con un solo guion.** `a -> b` truena. Van dos: `a --> b`.
- × **Olvidar el `end` del subgraph.** Cada `subgraph` necesita su `end`, y es el error que más tiempo quita porque el mensaje apunta a otra línea.
- × **Llamarle `end` a un nodo.** Es palabra reservada, así que `end[Final]` truena. Ponle otro nombre.
- ✓ **Los acentos y la ñ funcionan sin problema.** `a[Diseño de la aplicación]` dibuja perfecto, igual que los dos puntos y los guiones dentro del texto. No tienes que escribir sin acentos.

Actividad en clase · Tu sistema documentado

Se trabaja y se entrega hoy mismo

Con los contenedores y componentes que trajiste preparados de la sesión anterior, arma la documentación de tu propio sistema. No es un sistema nuevo: es el que vienes trabajando desde la Actividad 2.

Lo que vas a producir:

> 1. **Diagrama de contexto** en Mermaid. Tu sistema como una sola caja, las personas que lo usan y los sistemas de fuera con los que habla. Sin tecnología.

2. **Diagrama de contenedores** en Mermaid. Las piezas que se despliegan por separado, con su tecnología, y las flechas etiquetadas con cómo se comunican.

3. **Diagrama de componentes** en Mermaid, del contenedor donde viven tus reglas de negocio. Aquí van los módulos que ya definiste.

4. **Un ADR** con las cinco secciones, sobre la decisión de arquitectura que tomaste en la Actividad 2, ya sea monolito, monolito modular o microservicios. Escribe el contexto real de tu sistema, no el de la clínica.

Cómo se entrega:

1. Los tres diagramas van como texto de Mermaid en un archivo `doc/arch/diagramas.md` de tu repositorio, no como imagen.

2. El ADR va en `doc/arch/adr-001-nombre-de-tu-decision.md`, con el nombre en minúsculas y con guiones.

3. Sube los cambios al repositorio y entrega en la plataforma el enlace al repositorio, más una captura de uno de tus diagramas ya dibujado en mermaid.live, para que se vea que renderiza.

El criterio de revisión no es que esté bonito. Es que un compañero de otro equipo pueda abrir tu carpeta `doc/arch` y, sin preguntarte nada, entender qué construiste y por qué lo construiste así.

PARA PROFUNDIZAR

Nygard, M. (2011). *Documenting Architecture Decisions* · **cognitect.com**
El artículo original donde nace la plantilla. Es de 2011 y sigue siendo la referencia, lo cual dice bastante de lo bien resuelta que estaba.

Documentación de Mermaid · **mermaid.js.org**
La referencia completa de `flowchart`, con todas las formas de caja y tipos de flecha. Para cuando quieras algo que no está en la lista de arriba.

Editor en línea de Mermaid · **mermaid.live**
Donde vamos a trabajar hoy. No necesita cuenta ni instalación.



REFLEXIÓN FINAL

El código dice qué hace el sistema.
Solo tú puedes dejar escrito por qué lo hace así,
y solo mientras todavía te acuerdas.